

ATELIER PRATIQUE

Flask — Créer une API REST de gestion des tâches

Travail individuel ou en binôme

Objectif

Créer une API REST complète permettant de gérer des tâches (Tasks). Vous allez implémenter les opérations CRUD (Create, Read, Update, Delete) et apprendre à structurer une application Flask de manière progressive.

Partie 1 — Mise en place du projet

1. Créez un dossier nommé **flask_tasks_api**.
2. Dans ce dossier, créez un fichier **requirements.txt** contenant :

```
Flask==3.0.3
```

3. Installez les dépendances :

```
pip install -r requirements.txt
```

4. Créez un fichier **app.py** avec le code de base suivant :

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return "Bienvenue sur l'API Tasks !"

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

→ **Lancez l'application et vérifiez qu'elle fonctionne sur <http://localhost:5000>**

Partie 2 — Implémentation de l'API Tasks

Vous allez créer une API pour gérer des **tâches**. Chaque tâche possède les propriétés suivantes :

Champ	Type	Description
id	int	Identifiant unique généré automatiquement
titre	string	Titre de la tâche (obligatoire)
description	string	Description détaillée (optionnel)
done	bool	Indique si la tâche est terminée (défaut : false)
created_at	datetime	Date et heure de création

Stockage : Utilisez une liste Python en mémoire (TASKS = []).

Routes à implémenter

Méthode	Endpoint	Action
GET	/api/tasks	Récupérer la liste de toutes les tâches

GET	/api/tasks/<id>	Récupérer une tâche précise par son identifiant
POST	/api/tasks	Créer une nouvelle tâche
PUT	/api/tasks/<id>	Mettre à jour une tâche (ex: la marquer comme terminée)
DELETE	/api/tasks/<id>	Supprimer une tâche

Travail à réaliser (étapes recommandées)

Étape 1 — Seed data + GET /api/tasks

Ajoutez 3 tâches de démonstration au démarrage de l'application. Implémentez la route GET qui retourne toutes les tâches au format JSON.

Indice : Utilisez `jsonify()` et structurez la réponse avec les clés 'data' et 'count'.

Étape 2 — POST /api/tasks

Permettez de créer une nouvelle tâche. Le corps de la requête doit contenir au minimum le champ 'titre'. Générez automatiquement l'id et la date de création.

Indice : Utilisez `request.get_json()` pour lire les données envoyées par le client.

Étape 3 — GET /api/tasks/<id>

Récupérez une tâche spécifique. Si l'identifiant n'existe pas, retournez une réponse 404 avec un message d'erreur clair.

Étape 4 — PUT /api/tasks/<id>

Permettez de modifier une tâche (par exemple pour changer son statut 'done').

Étape 5 — DELETE /api/tasks/<id>

Supprimez une tâche. Gérez le cas où la tâche n'existe pas.

Partie 3 — Améliorations

Une fois le CRUD fonctionnel, ajoutez les améliorations suivantes :

- Validation : le champ 'titre' est obligatoire
- Gestion d'erreurs propre (400 pour les données invalides, 404 pour les ressources inexistantes)
- Filtrage : GET /api/tasks?done=true pour récupérer uniquement les tâches terminées (ou false)
- Bonus : Ajoutez un champ 'updated_at' qui se met à jour à chaque modification

Partie Bonus — Refactorisation (Structure plus propre)

Objectif : Améliorer l'organisation du code en séparant les responsabilités.

Consignes du bonus :

1. Créez un dossier **routes/** et un fichier **tasks_routes.py**
2. Créez un dossier **controllers/** et un fichier **task_controller.py**
3. Déplacez toute la logique métier dans le controller (fonctions `get_all_tasks`, `create_task`, etc.)
4. Dans **tasks_routes.py**, utilisez un **Blueprint** et appelez les fonctions du controller
5. Dans **app.py**, enregistrez le Blueprint avec `app.register_blueprint(...)`

→ Cette structure vous permettra plus facilement d'ajouter de nouvelles fonctionnalités par la suite.

Conseil : Testez chaque route avec Postman au fur et à mesure de son implémentation. C'est la meilleure façon de progresser et de détecter les erreurs rapidement.